

Ubuntu 24.04 安装 RealSense L515: librealsense v2.54.2 + RSUSB + Conda Python 3.10

1. 文档适用范围

本文档适用于以下环境：

项目	配置
操作系统	Ubuntu 24.04.4 LTS
Linux 内核	Linux 6.17.0-35-generic
CPU 架构	x86-64
相机	Intel RealSense L515
相机固件	1.5.8.1
USB 连接	USB 3.2
Conda 环境	<code>real_robot</code>
Python	3.10
librealsense	v2.54.2
后端	RSUSB / libusb
Python 包	源码编译的 <code>pyrealsense2</code>

本文档主要解决以下问题：

1. Ubuntu 24.04 和较新 Linux 内核无法安全使用旧版 RealSense DKMS。
2. L515 已停止维护，需要固定在保留 L515 支持的 librealsense 版本。
3. 默认 V4L2/XU 路径下，L515 的 Short Range 和 Digital Gain 可能读写失败。
4. PyPI 上的预编译 `pyrealsense2` 无法保证使用 RSUSB 后端。
5. librealsense v2.54.2 在 Ubuntu 24.04、GCC 13 下存在旧源码兼容性问题。

2. 最终采用的方案

使用以下组合：

```
Ubuntu 24.04
Linux 6.17
librealsense v2.54.2
FORCE_RSUSB_BACKEND=ON
BUILD_PYTHON_BINDINGS=ON
```

```
BUILD_SHARED_LIBS=OFF  
Conda Python 3.10
```

该方案具有以下特点：

- 保留 L515 支持。
- 通过 libusb 用户态后端访问相机。
- 绕过 Linux `uvcvideo` 的 XU ioctl 路径。
- 不安装 RealSense DKMS。
- 不修改或替换内核模块。
- 不执行 RealSense 内核 patch 脚本。
- Python 模块静态链接 librealsense，避免加载错误的系统版本。

librealsense v2.54.2 仍保留 L515 支持，但官方已经不再对 L515 进行完整验证。L515 最后一个完整验证版本是 v2.50.0。

3. 不推荐的安装方式

不要在 Ubuntu 24.04 上直接添加 Ubuntu 22.04 Jammy 的旧版 RealSense APT 软件源：

```
deb https://librealsense.realsenseai.com/Debian/apt-repo jammy main
```

不要安装：

```
sudo apt install librealsense2-dkms
```

不要执行：

```
./scripts/patch-realsense-ubuntu-lts.sh  
./scripts/patch-realsense-ubuntu-lts-hwe.sh
```

不要将以下命令作为正式安装方案：

```
pip install pyrealsense2==2.54.2.5684
```

Python 3.10 可以安装该 wheel，但无法控制 wheel 编译时使用的底层 USB 后端，因此不能保证绕过内核 XU 问题。

4. 检查系统信息

```
lsb_release -a  
hostnamectl  
uname -a
```

本机实测环境：

```
Ubuntu 24.04.4 LTS  
Linux 6.17.0-35-generic  
x86-64
```

5. 清理已有 RealSense 安装

5.1 激活 Conda 环境

```
conda activate real_robot
```

5.2 卸载已有 Python 包

```
python -m pip uninstall -y pyrealsense pyrealsense2
```

其中：

- `pyrealsense` 是旧版 librealsense v1 的社区封装。
- `pyrealsense2` 是 librealsense v2 的 Python 绑定。

5.3 检查系统包

```
dpkg -l | grep -i librealsense  
dkms status | grep -i realsense
```

如果安装过 APT 版本：

```
sudo apt-mark unhold  
librealsense2  
librealsense2-dev  
librealsense2-utils  
librealsense2-gl  
librealsense2-udev-rules
```

```
librealsense2-dkms
2>/dev/null || true

sudo apt purge -y 'librealsense2*'
sudo apt autoremove -y
```

删除可能存在的 RealSense 软件源：

```
sudo rm -f /etc/apt/sources.list.d/librealsense*
sudo apt update
```

再次确认：

```
dpkg -l | grep -i librealsense
dkms status | grep -i realsense
```

理想状态：

- 不存在 librealsense2-dkms。
- DKMS 中不存在 RealSense 模块。

6. 创建 Conda Python 3.10 环境

若环境已经存在，可跳过创建步骤。

```
conda create -n real_robot python=3.10 pip numpy -y
conda activate real_robot
```

更新基础 Python 工具：

```
python -m pip install --upgrade pip setuptools wheel
```

确认环境：

```
which python
python --version
echo "$CONDA_PREFIX"
```

预期输出类似：

```
/home/zhanghaoyang/miniconda3/envs/real_robot/bin/python  
Python 3.10.x  
/home/zhanghaoyang/miniconda3/envs/real_robot
```

7. 安装编译依赖

```
sudo apt update  
  
sudo apt install -y  
git  
cmake  
ninja-build  
build-essential  
gcc  
g++  
pkg-config  
libssl-dev  
libusb-1.0-0-dev  
libudev-dev
```

确认系统 libusb:

```
dpkg -L libusb-1.0-0-dev |  
grep -E 'libusb\.h$|libusb-1\.0\.so$'
```

预期:

```
/usr/include/libusb-1.0/libusb.h  
/usr/lib/x86_64-linux-gnu/libusb-1.0.so
```

检查 `pkg-config`:

```
pkg-config --modversion libusb-1.0  
pkg-config --cflags libusb-1.0  
pkg-config --libs libusb-1.0
```

本机实测输出:

```
1.0.27  
-I/usr/include/libusb-1.0  
-lusb-1.0
```

8. 下载 librealsense v2.54.2

```
mkdir -p ~/src
cd ~/src

git clone
  --branch v2.54.2
  --depth 1
  --recurse-submodules
  https://github.com/realsenseai/librealsense.git
  librealsense-2.54.2-rsusb

cd ~/src/librealsense-2.54.2-rsusb
```

确认版本：

```
git describe --tags --always
```

预期：

```
v2.54.2
```

9. 安装 RealSense udev 权限规则

执行前建议拔掉相机。

```
cd ~/src/librealsense-2.54.2-rsusb

sudo ./scripts/setup_udev_rules.sh
sudo udevadm control --reload-rules
sudo udevadm trigger
```

完成后重新插入 L515。

该脚本只设置 USB 设备权限，不修改 Linux 内核模块。

10. 修复 GCC 13 编译兼容性问题

librealsense v2.54.2 的以下文件使用了 `uint64_t`，但没有显式包含 `<stdint>`：

```
third-party/rsutils/include/rsutils/version.h
```

执行：

```
cd ~/src/librealsense-2.54.2-rsusb

if ! grep -q '^#include <stdint>$'
third-party/rsutils/include/rsutils/version.h; then
sed -i '/#include <string>/i #include <stdint>'
third-party/rsutils/include/rsutils/version.h
fi
```

检查：

```
head -n 12
third-party/rsutils/include/rsutils/version.h
```

应包含：

```
#include <stdint>
#include <string>
#include <iosfwd>
```

否则 GCC 13 可能出现：

```
error: 'uint64_t' does not name a type
note: 'uint64_t' is defined in header '<stdint>'
```

11. 配置系统编译器和 libusb

11.1 激活环境

```
conda activate real_robot
cd ~/src/librealsense-2.54.2-rsusb
```

11.2 删除旧构建目录

```
rm -rf build-rsusb-py310
```

CMake 会缓存编译器和依赖路径，因此切换编译器后必须删除旧构建目录。

11.3 清理 Conda 编译器环境变量

```
unset CC
unset CXX
unset CPP
unset CFLAGS
unset CXXFLAGS
unset CPPFLAGS
unset LDFLAGS
unset CMAKE_TOOLCHAIN_FILE
```

明确使用 Ubuntu 系统编译器：

```
export CC=/usr/bin/gcc
export CXX=/usr/bin/g++
```

确认：

```
"$CC" --version
"$CXX" --version
```

本机实测：

```
gcc 13.3.0
g++ 13.3.0
```

不要使用：

```
$CONDA_PREFIX/bin/x86_64-conda-linux-gnu-cc
$CONDA_PREFIX/bin/x86_64-conda-linux-gnu-c++
```

11.4 设置系统 libusb 路径

```
MULTIARCH="$(/usr/bin/gcc -print-multiarch)"

LIBUSB_LIB="/usr/lib/${MULTIARCH}/libusb-1.0.so"
LIBUSB_INC="/usr/include/libusb-1.0"

echo "$LIBUSB_LIB"
echo "$LIBUSB_INC"
```

检查文件：

```

test -f "$LIBUSB_LIB" || {
    echo "找不到 $LIBUSB_LIB"
    exit 1
}

test -f "$LIBUSB_INC/libusb.h" || {
    echo "找不到 $LIBUSB_INC/libusb.h"
    exit 1
}

```

11.5 获取 Conda Python 安装目录

```

PYTHON_INSTALL_DIR="$(
python - <<'PY'
import sysconfig
print(sysconfig.get_paths()["platlib"])
PY
)"

echo "$PYTHON_INSTALL_DIR"

```

预期：

```
/home/zhanghaoyang/miniconda3/envs/real_robot/lib/python3.10/site-packages
```

12. 配置 CMake

```

cmake -S . -B build-rsusb-py310 -G Ninja
-DMAKE_BUILD_TYPE=Release
-DMAKE_C_COMPILER=/usr/bin/gcc
-DMAKE_CXX_COMPILER=/usr/bin/g++
-DMAKE_INSTALL_PREFIX="$CONDA_PREFIX"
-DPYTHON_EXECUTABLE="$CONDA_PREFIX/bin/python"
-DPYTHON_INSTALL_DIR:PATH="$PYTHON_INSTALL_DIR"
-DLIBUSB_LIB:FILEPATH="$LIBUSB_LIB"
-DLIBUSB_INC:PATH="$LIBUSB_INC"
-DUSE_EXTERNAL_USB=OFF
-DFORCE_RSUSB_BACKEND=ON
-DBUILD_PYTHON_BINDINGS=ON
-DBUILD_SHARED_LIBS=OFF
-DBUILD_EXAMPLES=OFF
-DBUILD_GRAPHICAL_EXAMPLES=OFF
-DBUILD_GLSL_EXTENSIONS=OFF
-DBUILD_TOOLS=OFF

```

```
-DBUILD_UNIT_TESTS=OFF
-DCHECK_FOR_UPDATES=OFF
-DIMPORT_DEPTH_CAM_FW=OFF
```

关键参数说明：

参数	作用
<code>FORCE_RSUSB_BACKEND=ON</code>	使用 RSUSB/libusb 用户态后端
<code>BUILD_PYTHON_BINDINGS=ON</code>	编译 <code>pyrealsense2</code>
<code>BUILD_SHARED_LIBS=OFF</code>	静态链接 librealsense，避免版本冲突
<code>PYTHON_EXECUTABLE</code>	指定 Conda Python
<code>PYTHON_INSTALL_DIR</code>	指定 Conda 的 <code>site-packages</code>
<code>LIBUSB_LIB</code>	强制使用 Ubuntu 系统 libusb
<code>LIBUSB_INC</code>	指定系统 libusb 头文件
<code>USE_EXTERNAL_USB=OFF</code>	禁止下载并构建旧的内置 libusb
<code>BUILD_GRAPHICAL_EXAMPLES=OFF</code>	不编译 Viewer
<code>BUILD_TOOLS=OFF</code>	不编译额外命令行工具

配置日志中应看到：

```
Found usb: /usr/lib/x86_64-linux-gnu/libusb-1.0.so
using RS2_USE_LIBUVC_BACKEND
```

这里的：

```
RS2_USE_LIBUVC_BACKEND
```

表示正在使用 RSUSB/libuvc 用户态后端，不是默认的内核 V4L2 后端。

13. 检查 CMake 配置

13.1 检查编译器

```
grep -E '^CMAKE_(C|CXX)_COMPILER'
build-rsusb-py310/CMakeCache.txt
```

应使用：

```
/usr/bin/gcc  
/usr/bin/g++
```

13.2 检查关键选项

```
grep -E  
'^(FORCE_RSUSB_BACKEND|BUILD_PYTHON_BINDINGS|BUILD_SHARED_LIBS|  
USE_EXTERNAL_USB|PYTHON_INSTALL_DIR):'  
build-rsusb-py310/CMakeCache.txt
```

预期：

```
BUILD_PYTHON_BINDINGS:BOOL=ON  
BUILD_SHARED_LIBS:BOOL=OFF  
FORCE_RSUSB_BACKEND:BOOL=ON  
USE_EXTERNAL_USB:UNINITIALIZED=OFF  
PYTHON_INSTALL_DIR:PATH=.../lib/python3.10/site-packages
```

13.3 确认没有构建内置 libusb

```
test ! -d build-rsusb-py310/libusb-prefix &&  
echo "OK: 没有启用 external libusb"
```

预期：

```
OK: 没有启用 external libusb
```

如果构建时出现：

```
Performing download step (git clone) for 'libusb'
```

说明 CMake 没有使用系统 libusb，需要删除构建目录后重新配置。

14. 编译 librealsense 和 pyrealsense2

```
cmake --build build-rsusb-py310 -j8
```

成功时末尾类似：

```
Linking CXX shared library Release/pyrsutils...
Linking CXX shared library Release/pybackend2...
Linking CXX shared library Release/pyrealsense2...
Creating library symlink ...
```

类似以下警告通常可以忽略：

```
lto-wrapper: warning: using serial compilation
CMake Deprecation Warning
Policy CMP0148 is not set
-Wnonnull
-Wstringop-overflow
```

只要最终完成所有构建目标且没有 `FAILED` 或 `error:`，即可认为编译成功。

15. 安装到 Conda 环境

```
cmake --install build-rsusb-py310
```

不要使用：

```
sudo cmake --install build-rsusb-py310
```

否则可能在 Conda 环境中产生 root 所有权文件。

正确安装路径应包含：

```
$CONDA_PREFIX/lib/librealsense2.a
$CONDA_PREFIX/include/librealsense2/
$CONDA_PREFIX/lib/python3.10/site-packages/pyrealsense2...
$CONDA_PREFIX/lib/python3.10/site-packages/pybackend2...
$CONDA_PREFIX/lib/python3.10/site-packages/pyrsutils...
```

安装时可能出现：

```
/sbin/ldconfig.real: 无法创建临时缓存文件 /etc/ld.so.cache~: 权限不够
```

该提示可以忽略，原因是：

- 安装目标是 Conda 环境。
- `librealsense2` 被构建成静态库。
- Python 模块不依赖系统动态链接缓存。

16. 修复旧的错误安装目录

如果此前配置错误，Python 模块可能被安装到：

```
$CONDA_PREFIX/OFF
```

确认正确安装到 `site-packages` 后，可以删除：

```
rm -rf "$CONDA_PREFIX/OFF"
```

17. 验证 Python 模块

```
python - <<'PY'  
import sys  
import pyrealsense2 as rs  
  
print("Python:", sys.executable)  
print("pyrealsense2:", rs.__file__)  
PY
```

预期：

```
Python: .../envs/real_robot/bin/python  
pyrealsense2: .../envs/real_robot/lib/python3.10/site-packages/pyrealsense2...
```

18. 验证没有动态依赖系统 librealsense

```
MODULE_PATH="$(  
python -c 'import pyrealsense2 as rs; print(rs.__file__)'  
)"  
  
ldd "$MODULE_PATH" | grep -i realsense ||  
echo "OK: 未动态依赖系统 librealsense2.so"
```

预期：

OK: 未动态依赖系统 librealsense2.so

这说明当前 `pyrealsense2` 使用的是静态链接的 v2.54.2，而不是系统中其他版本的 `librealsense2.so`。

19. 验证 L515 设备识别

重新插拔相机后运行：

```
python - <<'PY'
import pyrealsense2 as rs

ctx = rs.context()
devices = ctx.query_devices()

print("Device count:", len(devices))

for dev in devices:
    print("Name:", dev.get_info(rs.camera_info.name))
    print("Serial:", dev.get_info(rs.camera_info.serial_number))
    print("Firmware:", dev.get_info(rs.camera_info.firmware_version))

    try:
        print(
            "USB:",
            dev.get_info(rs.camera_info.usb_type_descriptor)
        )
    except RuntimeError:
        pass
PY
```

本机实测结果：

```
Device count: 1
Name: Intel RealSense L515
Serial: f1382055
Firmware: 1.5.8.1
USB: 3.2
```

20. 验证 Short Range 和深度流

将以下整段复制到终端：

```

cat > /tmp/test_l515_short_range_runtime.py <<'PY'
import math
import time
import pyrealsense2 as rs

DEPTH_WIDTH = 1024
DEPTH_HEIGHT = 768
DEPTH_FPS = 30

SHORT_RANGE_PRESET = 5.0
WARMUP_FRAMES = 30
TEST_FRAMES = 100
DIGITAL_GAIN_READS = 10

def get_range_default(option_range):
    if hasattr(option_range, "default"):
        return getattr(option_range, "default")

    return getattr(option_range, "def")

def get_option_description(sensor, option, value):
    try:
        return sensor.get_option_value_description(option, value)
    except RuntimeError:
        return "Unknown"

ctx = rs.context()
devices = ctx.query_devices()

if len(devices) == 0:
    raise RuntimeError("No RealSense device found")

device = devices[0]

name = device.get_info(rs.camera_info.name)
serial = device.get_info(rs.camera_info.serial_number)
firmware = device.get_info(rs.camera_info.firmware_version)

try:
    usb_type = device.get_info(rs.camera_info.usb_type_descriptor)
except RuntimeError:
    usb_type = "Unknown"

print("=" * 60)
print("Device:", name)
print("Serial:", serial)

```

```

print("Firmware:", firmware)
print("USB:", usb_type)
print("=" * 60)

pipeline = rs.pipeline(ctx)
config = rs.config()

config.enable_device(serial)

config.enable_stream(
    rs.stream.depth,
    DEPTH_WIDTH,
    DEPTH_HEIGHT,
    rs.format.z16,
    DEPTH_FPS,
)

profile = None

try:
    profile = pipeline.start(config)

    active_device = profile.get_device()
    sensor = active_device.first_depth_sensor()

    if not sensor.supports(rs.option.visual_preset):
        raise RuntimeError("visual_preset is not supported")

    preset_before = sensor.get_option(rs.option.visual_preset)
    preset_before_name = get_option_description(
        sensor,
        rs.option.visual_preset,
        preset_before,
    )

    print(
        "Preset immediately after pipeline.start():",
        preset_before,
        preset_before_name,
    )

    print("\nSetting Short Range preset...")

    sensor.set_option(
        rs.option.visual_preset,
        SHORT_RANGE_PRESET,
    )

    time.sleep(0.8)

    preset_after = sensor.get_option(rs.option.visual_preset)

```

```

preset_after_name = get_option_description(
    sensor,
    rs.option.visual_preset,
    preset_after,
)

print(
    "Preset after setting:",
    preset_after,
    preset_after_name,
)

digital_gain = None

if sensor.supports(rs.option.digital_gain):
    digital_gain = sensor.get_option(
        rs.option.digital_gain
    )
    print("Digital Gain:", digital_gain)

laser_power = None
laser_default = None
laser_maximum = None

if sensor.supports(rs.option.laser_power):
    laser_range = sensor.get_option_range(
        rs.option.laser_power
    )

    laser_power = sensor.get_option(
        rs.option.laser_power
    )

    laser_default = get_range_default(laser_range)
    laser_maximum = laser_range.max

    print("Laser Power:", laser_power)
    print("Laser default:", laser_default)
    print("Laser maximum:", laser_maximum)

print(f"\nDiscarding {WARMUP_FRAMES} warm-up frames...")

for _ in range(WARMUP_FRAMES):
    pipeline.wait_for_frames(5000)

print(f"Collecting {TEST_FRAMES} test frames...")

previous_frame_number = None
non_monotonic_frames = 0

for index in range(TEST_FRAMES):

```

```

frames = pipeline.wait_for_frames(5000)
depth = frames.get_depth_frame()

if not depth:
    raise RuntimeError(
        f"No depth frame at iteration {index + 1}"
    )

frame_number = depth.get_frame_number()

if (
    previous_frame_number is not None
    and frame_number <= previous_frame_number
):
    non_monotonic_frames += 1

previous_frame_number = frame_number

if (index + 1) % 20 == 0:
    print(
        f"Frame {index + 1}: "
        f"{depth.get_width()}x{depth.get_height()}, "
        f"number={frame_number}"
    )

print("\nDigital Gain stability test:")

successful_gain_reads = 0

if sensor.supports(rs.option.digital_gain):
    for index in range(DIGITAL_GAIN_READS):
        try:
            value = sensor.get_option(
                rs.option.digital_gain
            )

            successful_gain_reads += 1
            print(
                f" Read {index + 1}: OK, value={value}"
            )

        except RuntimeError as error:
            print(
                f" Read {index + 1}: FAILED, {error}"
            )

        time.sleep(0.2)

final_preset = sensor.get_option(
    rs.option.visual_preset
)

```

```

final_preset_name = get_option_description(
    sensor,
    rs.option.visual_preset,
    final_preset,
)

final_gain = None
final_laser = None

if sensor.supports(rs.option.digital_gain):
    final_gain = sensor.get_option(
        rs.option.digital_gain
    )

if sensor.supports(rs.option.laser_power):
    final_laser = sensor.get_option(
        rs.option.laser_power
    )

print("\n" + "=" * 60)
print("Final preset:", final_preset, final_preset_name)
print("Final Digital Gain:", final_gain)
print("Final Laser Power:", final_laser)
print(
    "Digital Gain successful reads:",
    f"{successful_gain_reads}/{DIGITAL_GAIN_READS}",
)
print("Non-monotonic frame count:", non_monotonic_frames)

preset_reports_short_range = math.isclose(
    final_preset,
    SHORT_RANGE_PRESET,
    abs_tol=1e-6,
)

parameter_signature_matches = False

if (
    final_gain is not None
    and final_laser is not None
    and laser_default is not None
):
    parameter_signature_matches = (
        math.isclose(final_gain, 2.0, abs_tol=1e-6)
        and math.isclose(
            final_laser,
            laser_default,
            abs_tol=1e-6,
        )
    )

```

```

stream_ok = (
    previous_frame_number is not None
    and non_monotonic_frames == 0
)

gain_ok = (
    not sensor.supports(rs.option.digital_gain)
    or successful_gain_reads == DIGITAL_GAIN_READS
)

short_range_ok = (
    preset_reports_short_range
    or parameter_signature_matches
)

print("\nResult:")
print(
    " Depth streaming:",
    "PASS" if stream_ok else "FAIL",
)
print(
    " Digital Gain access:",
    "PASS" if gain_ok else "FAIL",
)
print(
    " Short Range configuration:",
    "PASS" if short_range_ok else "FAIL",
)

if not stream_ok:
    raise RuntimeError(
        "Depth streaming validation failed"
    )

if not gain_ok:
    raise RuntimeError(
        "Digital Gain stability validation failed"
    )

if not short_range_ok:
    raise RuntimeError(
        "Short Range configuration validation failed"
    )

print("\nRuntime Short Range test passed")

finally:
    if profile is not None:
        try:
            pipeline.stop()

```

```
        print("Pipeline stopped cleanly")
    except RuntimeError as error:
        print("Pipeline stop warning:", error)
PY

python /tmp/test_l515_short_range_runtime.py
```

本机实测通过结果：

```
Preset after setting: 5.0 Short Range
Digital Gain: 2.0
Laser Power: 100.0

Frame 100: 1024x768

Final preset: 5.0 Short Range
Final Digital Gain: 2.0
Digital Gain successful reads: 10/10
Non-monotonic frame count: 0

Depth streaming: PASS
Digital Gain access: PASS
Short Range configuration: PASS

Runtime Short Range test passed
Pipeline stopped cleanly
```

21. 正式程序中的正确初始化顺序

测试表明，`pipeline.start()` 可能会将启动前设置的 Short Range 重置为 Low Ambient Light。

因此正式程序必须采用以下顺序：

```
创建 context
→ 使用该 context 创建 pipeline
→ pipeline.start()
→ 从 pipeline profile 获取活动 device
→ 获取活动 depth sensor
→ 设置 Short Range
→ 丢弃预热帧
→ 开始正式采集
```

推荐代码：

```

import time
import pyrealsense2 as rs

def start_l515_short_range(
    width: int = 1024,
    height: int = 768,
    fps: int = 30,
    warmup_frames: int = 30,
):
    ctx = rs.context()
    devices = ctx.query_devices()

    if len(devices) == 0:
        raise RuntimeError("No RealSense device found")

    device = devices[0]
    serial = device.get_info(
        rs.camera_info.serial_number
    )

    pipeline = rs.pipeline(ctx)
    config = rs.config()

    config.enable_device(serial)
    config.enable_stream(
        rs.stream.depth,
        width,
        height,
        rs.format.z16,
        fps,
    )

    profile = pipeline.start(config)

    active_device = profile.get_device()
    depth_sensor = active_device.first_depth_sensor()

    depth_sensor.set_option(
        rs.option.visual_preset,
        5.0,
    )

    time.sleep(0.5)

    preset = depth_sensor.get_option(
        rs.option.visual_preset
    )

    if preset != 5.0:

```

```

        pipeline.stop()
        raise RuntimeError(
            f"Failed to enable Short Range, preset={preset}"
        )

    for _ in range(warmup_frames):
        pipeline.wait_for_frames(5000)

    return pipeline, profile, depth_sensor

```

调用：

```

pipeline, profile, depth_sensor = (
    start_l515_short_range()
)

try:
    while True:
        frames = pipeline.wait_for_frames(5000)
        depth_frame = frames.get_depth_frame()

        if not depth_frame:
            continue

        # 在此处理深度图或点云

finally:
    pipeline.stop()

```

22. 常见报错及解决方案

22.1 fatal error: config.h: No such file or directory

典型输出：

```
libusb/core.c: fatal error: config.h: No such file or directory
```

原因：

- CMake 没有找到 Ubuntu 系统 libusb。
- librealsense 回退到旧的内置 libusb 构建。
- 旧构建脚本没有正确生成 `config.h`。

解决：

1. 安装 `libusb-1.0-0-dev`。

2. 使用 `/usr/bin/gcc` 和 `/usr/bin/g++`。
3. 显式设置 `LIBUSB_LIB` 和 `LIBUSB_INC`。
4. 设置 `USE_EXTERNAL_USB=OFF`。
5. 删除旧构建目录后重新配置。

22.2 `uint64_t` does not name a type

典型输出：

```
error: 'uint64_t' does not name a type
```

解决：

```
sed -i '/#include <string>/i #include <stdint>'
third-party/rsutils/include/rsutils/version.h
```

然后继续构建：

```
cmake --build build-rsusb-py310 -j8
```

不需要删除整个构建目录。

22.3 Python 模块安装到 `$CONDA_PREFIX/OFF`

原因：

```
PYTHON_INSTALL_DIR=OFF
```

解决：

```
PYTHON_INSTALL_DIR="$(
python - <<'PY'
import sysconfig
print(sysconfig.get_paths()["platlib"])
PY
)"

cmake -S . -B build-rsusb-py310
-DPYTHON_INSTALL_DIR:PATH="$PYTHON_INSTALL_DIR"

cmake --install build-rsusb-py310
```

确认成功后：

```
rm -rf "$CONDA_PREFIX/OFF"
```

22.4 ldconfig 权限不足

输出：

```
/sbin/ldconfig.real: 无法创建临时缓存文件
```

可以忽略。

不要因此使用：

```
sudo cmake --install
```

22.5 RuntimeError: failed to set power state

常见原因是创建了两个不同的 RealSense Context：

```
pipeline = rs.pipeline()  
ctx = rs.context()
```

随后通过一个 Context 启动流，通过另一个 Context 获取 Sensor。

正确做法：

```
ctx = rs.context()  
pipeline = rs.pipeline(ctx)  
profile = pipeline.start(config)  
  
device = profile.get_device()  
sensor = device.first_depth_sensor()
```

流启动后只使用 `profile.get_device()` 返回的活动设备。

22.6 Short Range 启动后变成 Low Ambient Light

现象：

Preset before streaming: Short Range
Preset immediately after pipeline.start(): Low Ambient Light

解决: 在 `pipeline.start()` 之后重新设置 Short Range。

```
profile = pipeline.start(config)

sensor = profile.get_device().first_depth_sensor()

sensor.set_option(
    rs.option.visual_preset,
    5.0,
)
```

22.7 No module named pyrealsense2

检查:

```
python -c 'import sys; print(sys.executable)'
find "$CONDA_PREFIX" -name 'pyrealsense2*.so'
```

确认模块位于:

```
$CONDA_PREFIX/lib/python3.10/site-packages/
```

22.8 Device count: 0

检查 USB:

```
lsusb | grep -i 8086
lsusb -t
```

确认设备使用 USB 3.x, 而不是:

```
480M
```

重新安装 udev 规则:

```
sudo ./scripts/setup_udev_rules.sh
sudo udevadm control --reload-rules
sudo udevadm trigger
```

然后重新插拔相机。

同时关闭其他可能占用相机的程序：

```
killall -f realsense-viewer 2>/dev/null || true
killall -f realsense 2>/dev/null || true
```

23. 当前安装的成功判据

本机已经满足以下全部条件：

```
pyrealsense2 导入成功
模块位于 Conda Python 3.10 的 site-packages
未动态依赖系统 librealsense2.so
L515 成功枚举
固件为 1.5.8.1
USB 为 3.2
Short Range 设置成功
Digital Gain 连续读取 10/10 成功
1024×768@30 Hz 连续采集 100 帧成功
帧号单调递增
Pipeline 正常停止
```

因此可以确认：

```
librealsense v2.54.2
+ RSUSB 后端
+ Conda Python 3.10
+ Intel RealSense L515
```

已经安装和运行成功。

24. 后续维护注意事项

1. 不要通过 pip 再安装其他版本的 `pyrealsense2`，否则可能覆盖当前源码版本。
2. 不要安装系统 APT 版 `librealsense2`，避免版本冲突。

3. 不要安装 `librealsense2-dkms`。
4. 不要执行 RealSense 内核 patch 脚本。
5. 重建 Conda 环境后，需要重新编译和安装 Python 绑定。
6. 当前 `.so` 对应 Python 3.10，不能直接复制给 Python 3.11 或 Python 3.12 使用。
7. 正式程序应在 `pipeline.start()` 后设置 Short Range。
8. 相机应直接连接主板 USB 3.x 接口，尽量避免廉价 USB Hub 和前置面板接口。
9. 同一时间只允许一个 Viewer、ROS 节点或 Python 进程访问相机。
10. 采集开始前建议丢弃约 30 帧预热帧。